SQL Exercise: CRUD, Constraints, and Queries - Answers

Part 1: Answers

Answers for creating tables and constraints are as follows:

1. Customers Table:
   customer_id INTEGER PRIMARY KEY AUTOINCREMENT
   first_name TEXT NOT NULL
   last_name TEXT NOT NULL
   email TEXT UNIQUE
   phone_number TEXT
   created_at DATETIME DEFAULT CURRENT_TIMESTAMP

2. Products Table:
   product_id INTEGER PRIMARY KEY AUTOINCREMENT
   product_name TEXT NOT NULL
   price REAL NOT NULL CHECK(price > 0)
   stock_quantity INTEGER NOT NULL CHECK(stock_quantity >= 0)

3. Orders Table:
   order_id INTEGER PRIMARY KEY AUTOINCREMENT
   customer_id INTEGER
   order_date DATETIME DEFAULT CURRENT_TIMESTAMP
   total_amount REAL CHECK(total_amount >= 0)
   FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)

4. OrderDetails Table:
   order_detail_id INTEGER PRIMARY KEY AUTOINCREMENT
   order_id INTEGER
   product_id INTEGER
   quantity INTEGER CHECK(quantity > 0)
   price REAL NOT NULL
   FOREIGN KEY (order_id) REFERENCES Orders(order_id)
   FOREIGN KEY (product_id) REFERENCES Products(product_id)

Part 2: Answers

1. Retrieve all customers with their email addresses:

```
SELECT first_name, last_name, email FROM Customers;
```

2. Retrieve all orders with a total amount greater than 1000:

```
SELECT * FROM Orders WHERE total_amount > 1000;
```

3. Retrieve all products ordered by price in descending order:

```
SELECT * FROM Products ORDER BY price DESC;
```

4. Retrieve all orders along with customer information:

```
SELECT Orders.order_id, Customers.first_name, Customers.last_name, Orders.total_amount
FROM Orders
INNER JOIN Customers ON Orders.customer_id = Customers.customer_id;
```

5. Retrieve the total sales (quantity * price) for each product. Only show products with total sales greater than 1000:

```
SELECT Products.product_name, SUM(OrderDetails.quantity * OrderDetails.price) AS
total_sales
FROM OrderDetails
INNER JOIN Products ON OrderDetails.product_id = Products.product_id
GROUP BY Products.product_name
HAVING SUM(OrderDetails.quantity * OrderDetails.price) > 1000;
```

6. Retrieve customers whose last name starts with ‘S’:

```
SELECT * FROM Customers WHERE last_name LIKE 'S%';
```

7. Retrieve the total number of orders placed by each customer:

```
SELECT Customers.first_name, Customers.last_name, COUNT(Orders.order_id) AS
```

```
total_orders
FROM Customers
INNER JOIN Orders ON Customers.customer_id = Orders.customer_id
GROUP BY Customers.customer_id;
```

8. Retrieve all customers who have ordered a product with a price greater than 500:

```
SELECT DISTINCT Customers.first_name, Customers.last_name
FROM Customers
INNER JOIN Orders ON Customers.customer_id = Orders.customer_id
INNER JOIN OrderDetails ON Orders.order_id = OrderDetails.order_id
INNER JOIN Products ON OrderDetails.product_id = Products.product_id
WHERE Products.price > 500;
```

9. Retrieve products that have been ordered more than once, including product details:

```
SELECT Products.product_name, SUM(OrderDetails.quantity) AS total_ordered
FROM OrderDetails
INNER JOIN Products ON OrderDetails.product_id = Products.product_id
GROUP BY Products.product_name
HAVING COUNT(OrderDetails.order_id) > 1;
```